

A-RES Computer Engineering Technical Report

This report explains the role of the Computer Engineering team in the A-RES project, the technical system design, data sources, model strategy, validation plan, sensor relationships, and demonstration approach. The text is written so that it can be directly integrated into the project report.

1. Computer Engineering Team Role

The main responsibility of the Computer Engineering team in the A-RES project is to design the software system that collects, processes, and transforms sensor and camera data from buildings into emergency response priorities. Within this scope, our team is responsible for IoT communication, AI-supported occupancy estimation, data analysis, urgency score calculation, dashboard development, and providing structured data for the Industrial Engineering optimization team.

The main objective of the Computer Engineering part of A-RES can be summarized as follows:

Building sensor data + occupancy data + hazard data
-> central software system
-> urgency score
-> emergency response priority

Therefore, our task is not only to run an artificial intelligence model. The main task is to transform data from multiple sources into a meaningful decision-support system for post-earthquake emergency response.

2. Project Scope and Technical Positioning

A-RES is not an earthquake prediction system. It does not attempt to predict when an earthquake will occur before it happens. It is also not designed as a structural engineering diagnosis system that can definitively declare whether a building is safe or unsafe.

Our technical approach is to build a realistic and applicable decision-support system. A-RES uses post-earthquake data to calculate which buildings require more urgent emergency response. This decision is based on shaking intensity, estimated number of occupants, fire/smoke/gas risks, and the vulnerability level of the building.

This approach is important because one of the biggest problems after a disaster is the lack of reliable and timely information. If emergency teams can quickly understand which buildings still have people inside, which buildings may have fire or gas leakage, and which buildings experienced stronger shaking, rescue resources can be directed more effectively.

3. Data Sources: Where We Get the Data

The data used in A-RES can be divided into two main groups:

1. Real-world reference data
2. Simulation data generated for the MVP and demonstration

3.1 Earthquake Data

The primary source for earthquake data is the USGS ComCat data service. USGS provides global earthquake events with information such as magnitude, depth, location, and time. In this project, this data is used for earthquake analysis, magnitude and depth distribution studies, and scenario design.

For the Japan-based scenario, J-SHIS and NIED K-NET / KiK-net were also investigated. These are strong reference sources for seismic hazard maps and ground motion records in Japan. However, if access or data format issues occur, USGS data and simulation data are sufficient for the MVP stage.

The data sources used or investigated are listed below:

Data Source	Data Type	Project Use
USGS ComCat	Earthquake magnitude, depth, location, time	Earthquake analysis and scenario design
J-SHIS	Japan seismic hazard map information	Japan-based risk context
NIED K-NET / KiK-net	Strong-motion acceleration records	PGA and shaking reference
COCO Dataset	Person class for object detection	YOLO model pre-training source
MOT / PETS datasets	Pedestrian tracking videos	Optional occupancy counting validation
Synthetic A-RES data	Simulated building states	Demo, dashboard, and optimization input

3.2 Building and Sensor Data

Since real building sensors have not yet been physically deployed, building data is simulated during the MVP stage. For each building, the following fields are generated:

Data Field	Meaning
building_id	Unique building ID
city	Tokyo or Istanbul scenario
building_type	Hospital, school, residence, factory, tower
occupancy	Estimated number of people inside
pga	Peak Ground Acceleration
vulnerability	Building vulnerability proxy
smoke_detected	Smoke/fire signal
gas_detected	Gas leak signal
urgency_score	Final emergency priority score

This data is prepared in `data/optimization_input_template.csv` as a sample input file for the Industrial Engineering team's optimization model.

4. How the Sensors Detect Building Conditions

In A-RES, each building is represented by an Integrated Sensor Hub. This sensor hub measures the building's condition and sends the data to the central system.

The planned sensors are:

Sensor	What It Detects	Why It Matters
Accelerometer, e.g. MPU6050 / ADXL345	Vibration and acceleration	Measures shaking intensity and estimates PGA
MQ-2	Smoke / flammable gas	Detects fire or combustible gas risk
MQ-7	Carbon monoxide	Detects dangerous gas after fire or leakage
DS18B20 / BME280	Temperature, humidity, pressure	Supports fire and environmental anomaly detection
Camera	Person movement at the entrance	Estimates anonymous occupancy

4.1 Accelerometer Logic

The accelerometer measures how much the building shakes during an earthquake. From this signal, Peak Ground Acceleration (PGA) can be estimated. As the PGA value increases, the building is considered to have experienced stronger shaking.

Example interpretation:

PGA Range	Interpretation
< 0.02g	Normal / very low shaking
0.05g - 0.10g	Light to moderate shaking
0.20g - 0.35g	Strong shaking
> 0.50g	Severe shaking

This value alone does not mean that the building has collapsed. However, when high PGA is combined with high building vulnerability and high occupancy, the emergency response priority increases.

4.2 Smoke, Fire, and Gas Logic

Smoke and gas sensors are used to detect secondary hazards after an earthquake. Fire, gas leakage, or carbon monoxide risk may occur after seismic events. These risks may require urgent intervention even if the building has not suffered complete structural collapse.

For this reason, smoke/fire and gas data have separate weights in the urgency score. For example, a building with moderate shaking, many occupants, and detected gas leakage should be prioritized based on the combined risk, not only the shaking value.

4.3 Occupancy Detection Logic

The number of people inside a building is estimated by counting people who enter and exit through the doorway. For this purpose, YOLO11n is used for person detection, ByteTrack is used for tracking, and a virtual line-crossing method is used to determine entry and exit events.

The system follows this logic:

```
Person detected -> Track ID assigned -> Person crosses virtual line
    -> Entry or exit event
    -> Current occupancy updated
```

No face recognition or identity detection is performed. The system only counts how many people enter or leave the building.

5. Sensor Fusion: How Sensor Relationships Are Built

One of the most important aspects of A-RES is that it does not rely on a single sensor. Data from different sensors is combined to produce a more meaningful risk evaluation. This can be described as a sensor fusion approach.

Examples:

Scenario	Interpretation
High PGA + high occupancy	Many people may be at risk after strong shaking
Medium PGA + gas detected	Secondary hazard increases urgency
Low PGA + fire detected	Fire response may still be needed
High PGA + old building + many occupants	Rescue priority becomes high
Missing or stale data	Confidence decreases and the score may be cautiously boosted

These relationships are implemented through a rule-based urgency score formula. For the MVP stage, this approach is more suitable than a black-box ML model because the reason behind each decision can be explained. For example, if a building is classified as CRITICAL, the dashboard can show whether this was caused by high PGA, high occupancy, fire/gas detection, or building vulnerability.

6. Urgency Score Calculation

The urgency score is calculated between 0 and 100. This score indicates which building should receive emergency response first.

The components used in the formula are:

Component	Weight
Shaking severity / PGA	30%
Occupancy	25%
Fire / smoke	20%
Building vulnerability	15%
Gas leak	10%

Priority levels:

Score Range	Priority
0-30	LOW
31-60	MEDIUM
61-80	HIGH
81-100	CRITICAL

This score does not declare a building to be definitively safe or unsafe. It only helps allocate limited emergency resources more logically.

7. AI / ML Strategy: How We Train or Use Models

There are two AI-related approaches in the project:

1. A computer vision model for occupancy estimation
2. An optional machine learning model for urgency score approximation

7.1 Occupancy Counting Model

YOLO11n is used for occupancy counting. YOLO11n is already a pre-trained object detection model, and the COCO dataset includes the `person` class. Therefore, for the MVP stage, we do not need to train a new model from scratch.

This is an important decision because training a model from scratch would require a large number of labeled images, GPU resources, and additional time. Within the project timeline, the most practical approach is to use a reliable pre-trained model and integrate it into the A-RES use case.

The approach is:

```
Pre-trained YOLO11n model
-> detect person class
-> ByteTrack tracking
-> virtual line crossing
-> entry/exit count
```

7.2 Do We Train a New Model?

For the MVP, we do not train a new computer vision model. Instead, we use pre-trained YOLO11n. If accuracy is insufficient in future stages, the model can be fine-tuned with CrowdHuman,

MOT17/MOT20, or our own doorway videos.

For the urgency score, machine learning is optional. The main decision mechanism is currently the rule-based formula. However, a Random Forest or another tabular ML model can be trained on a synthetic dataset to imitate or compare against the rule-based score.

In that case, the training process would be:

- 1. Simulate different building scenarios.
- 2. Generate features such as PGA, occupancy, vulnerability, smoke, and gas.
- 3. Calculate the rule-based urgency score.
- 4. Train the ML model to predict the score from these inputs.
- 5. Compare the ML output with the rule-based score.

However, the main claim in the report should be clear: the MVP decision mechanism of A-RES is an explainable rule-based scoring system, while ML can be used as a supporting or future enhancement.

8. Validation: How We Validate the System

We plan to validate the system at three levels:

- 1. Unit tests
- 2. Integration tests
- 3. Real video / manual count validation

8.1 Unit Tests

Unit tests check whether the urgency score function behaves as expected. For example:

	Test Scenario		Expected Result	
	--- ---			
	Empty safe building		LOW priority	
	High PGA + high occupancy + fire/gas		CRITICAL priority	
	Old or stale data		Score should increase cautiously	

These tests are prepared to run automatically in GitHub Actions CI.

8.2 Integration Tests

The integration test evaluates sensor and occupancy payloads together. The goal is to check whether the system can build the latest building state and calculate the correct urgency score when both camera and sensor data arrive.

Example flow:

Sensor payload arrives
Occupancy payload arrives
Server combines latest state
Urgency score is calculated
Dashboard receives updated priority

8.3 Occupancy Validation with Phone Videos

The most important validation for occupancy counting will be performed using real video. A phone will be mounted at a doorway, and short videos will be recorded.

Test scenarios:

	Test		Description	
	--- ---			
	Single entry		One person enters	
	Single exit		One person exits	
	Two people		Two people pass close together	
	Turn back		A person approaches and turns back	
	Empty frame		No false count should occur	

For each video, manual counting will be performed and compared with the system output.

Example table:

	Video		Manual Count		System Count		Error	
	--- ---		----		----		----	
	entry_01.mp4		+1		+1		0	
	exit_01.mp4		-1		-1		0	
	two_people.mp4		+2		+2 or +1		0 or 1	

This table will show the accuracy of the system in the final report.

9. Demo Strategy: Phone Camera vs Laptop Processing

At first, we considered integrating the camera directly into the mobile application. In this approach, the phone would be placed near the doorway and the mobile app would open the camera to count people. However, after technical evaluation, we selected a more reliable approach for the MVP.

The updated demo approach is:

```
Phone records doorway video
-> laptop runs YOLO11n + ByteTrack
-> laptop publishes MQTT occupancy event
-> FastAPI server receives data
-> dashboard updates building priority
```

The reasons for this change are:

- 1. Running YOLO directly on a mobile device introduces higher integration risk.
- 2. Building a real-time camera + ML pipeline inside Flutter may exceed the project timeline.
- 3. Running the model on a laptop is easier to debug and demonstrate.
- 4. The same technical idea can be proven more reliably during the demo.
- 5. In the future, the laptop can be replaced with a Raspberry Pi, Jetson Nano, or mini PC.

Therefore, the mobile application is positioned as an emergency dashboard client for the MVP. Camera processing will be demonstrated on the laptop using the edge-device logic.

10. Communication Design: MQTT and Central Server

The sensors and camera system send data to the central server using MQTT. MQTT was selected because it is lightweight, fast, and suitable for IoT publish/subscribe communication.

Example occupancy message:

```
{
  "building_id": "JP-001",
  "camera_id": "CAM-001",
  "timestamp": "2026-05-10T12:00:00Z",
  "seq": 15,
  "event_type": "count_delta",
  "delta": 1,
  "direction": "ENTRY",
  "current_count": 47,
  "confidence": 0.95
}
```

Example sensor message:

```
{
  "building_id": "JP-001",
  "hub_id": "HUB-001",
  "timestamp": "2026-05-10T12:00:00Z",
  "seq": 42,
  "pga": 0.32,
  "smoke_detected": false,
  "gas_detected": true,
  "vulnerability": 0.4
}
```

The FastAPI server receives these messages, updates the building state, calculates the urgency score, and sends the updated result to the dashboard.

11. Data for the Optimization Team

The role of the Industrial Engineering team is to optimize the allocation of limited emergency resources among buildings. For this purpose, the Computer Engineering team prepared an input dataset.

File:

```
data/optimization_input_template.csv
```

This file includes sample buildings from both Tokyo and Istanbul scenarios. For each building, it includes fields such as urgency score, priority, required rescue units, required fire units, required medical units, and accessibility score.

The IE team can use this data to build a model such as:

```
Minimize total weighted response time
Subject to limited rescue/fire/medical team availability
```

Prioritize high urgency buildings first

As a result, A-RES becomes not only a data visualization system, but also a decision-support infrastructure for emergency resource allocation.

12. GitHub and CI/CD Work

The project has been uploaded to GitHub. The repository includes the README file, technical documentation, dashboard, backend, edge scripts, tests, and sample data files.

A lightweight GitHub Actions workflow has been added for CI/CD. This workflow runs tests after each push or pull request.

The main tested points are:

1. Does the urgency score produce the correct priority?
2. Does a critical scenario reach the CRITICAL level?
3. Does stale or low-confidence data cautiously increase the score?
4. Does the optimization CSV file have the correct columns and valid value ranges?

The YOLO model is not executed inside CI because it is a heavy dependency and may make GitHub Actions unnecessarily slow or fragile. YOLO validation will be performed with real video tests.

13. Current Deliverables

The main outputs prepared by the Computer Engineering team are:

Deliverable	File
GitHub README	`README.md`
System architecture	`MD_FILES/ARCHITECTURE.md`
Occupancy system design	`MD_FILES/OCCUPANCY_SYSTEM.md`
ML strategy	`MD_FILES/ML_STRATEGY.md`
Urgency score design	`MD_FILES/URGENCY_SCORE.md`
GitHub repository selection	`MD_FILES/GITHUB_REPOSITORY_SELECTION.md`
Optimization data package	`MD_FILES/OPTIMIZATION_DATA_PACKAGE.md`
Full CE technical report in English	`MD_FILES/COMPUTER_ENGINEERING_FULL_REPORT_EN.md`
Full CE technical report in Turkish	`MD_FILES/COMPUTER_ENGINEERING_FULL_REPORT_TR.md`
Optimization sample data	`data/optimization_input_template.csv`
Edge occupancy counter	`src/edge/occupancy_counter.py`
Simulation script	`src/edge/simulator.py`
Urgency engine	`src/server/urgency_engine.py`
CI tests	`tests/`

14. Next Steps

The next steps for the Computer Engineering team are:

1. Record doorway entry/exit videos using a phone camera.
2. Compare YOLO + ByteTrack output with manual counting.
3. Display real test output on the dashboard.
4. Record an end-to-end demo video for MQTT -> FastAPI -> dashboard.
5. Finalize the optimization CSV format with the Industrial Engineering team.
6. Support the financial, economic, and feasibility analysis sections with technical cost and feasibility information.

15. Summary

This week, the Computer Engineering team made the software and data infrastructure of A-RES more concrete. We designed how data from sensors and the camera system will be collected, processed, transformed into an urgency score, and transferred to the dashboard and optimization components.

The most important technical decisions are:

- A-RES will focus on post-earthquake emergency response prioritization instead of earthquake prediction.
- Pre-trained YOLO11n + ByteTrack will be used for occupancy estimation.
- The phone camera will be used as a video/test source, not as the main mobile ML processing device.
- Model processing will be performed on a laptop for the MVP.
- Sensor data will be sent to a central FastAPI server using MQTT.
- The urgency score will be calculated using an explainable rule-based formula.
- Optimization-ready CSV data will be provided to the Industrial Engineering team.
- GitHub documentation, README, CI tests, and technical reports have been prepared.

These outputs show that A-RES is moving beyond the concept stage and toward a testable MVP system.